



Sign Out

Account

 Function Approximation

25 min read

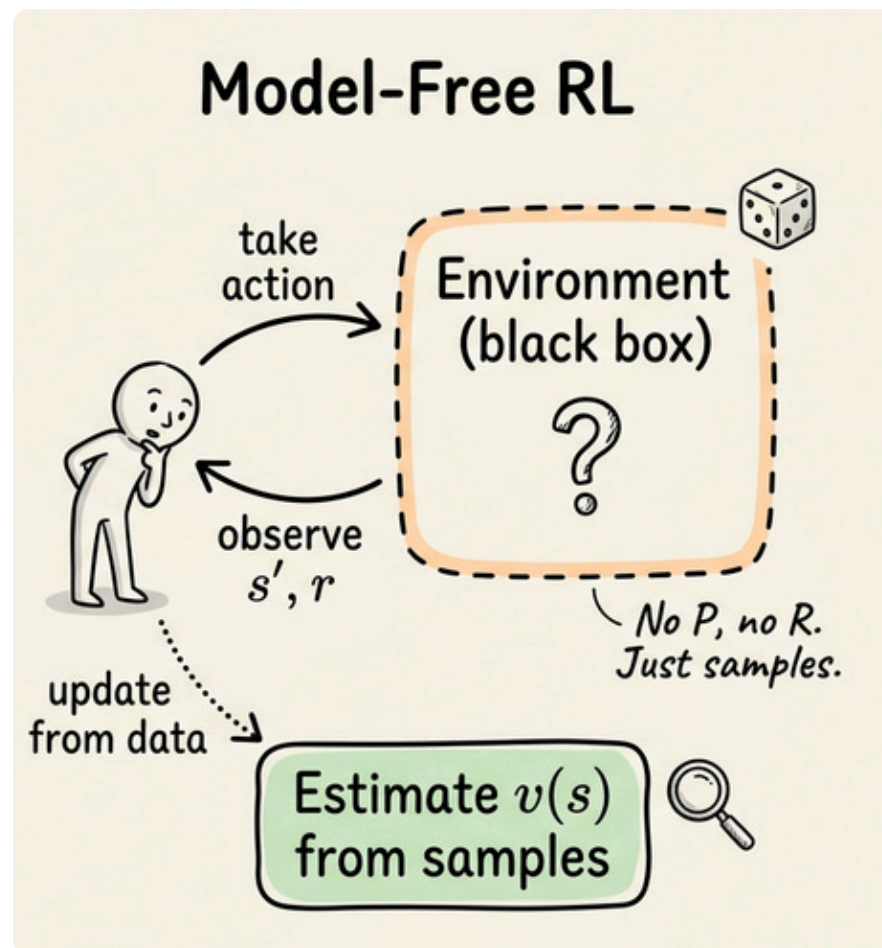
Function Approximation

RL Part 5: From tables to parameterized value functions.

Recap

In the previous chapter, we entered the model-free setting, where the agent learns purely from interaction.

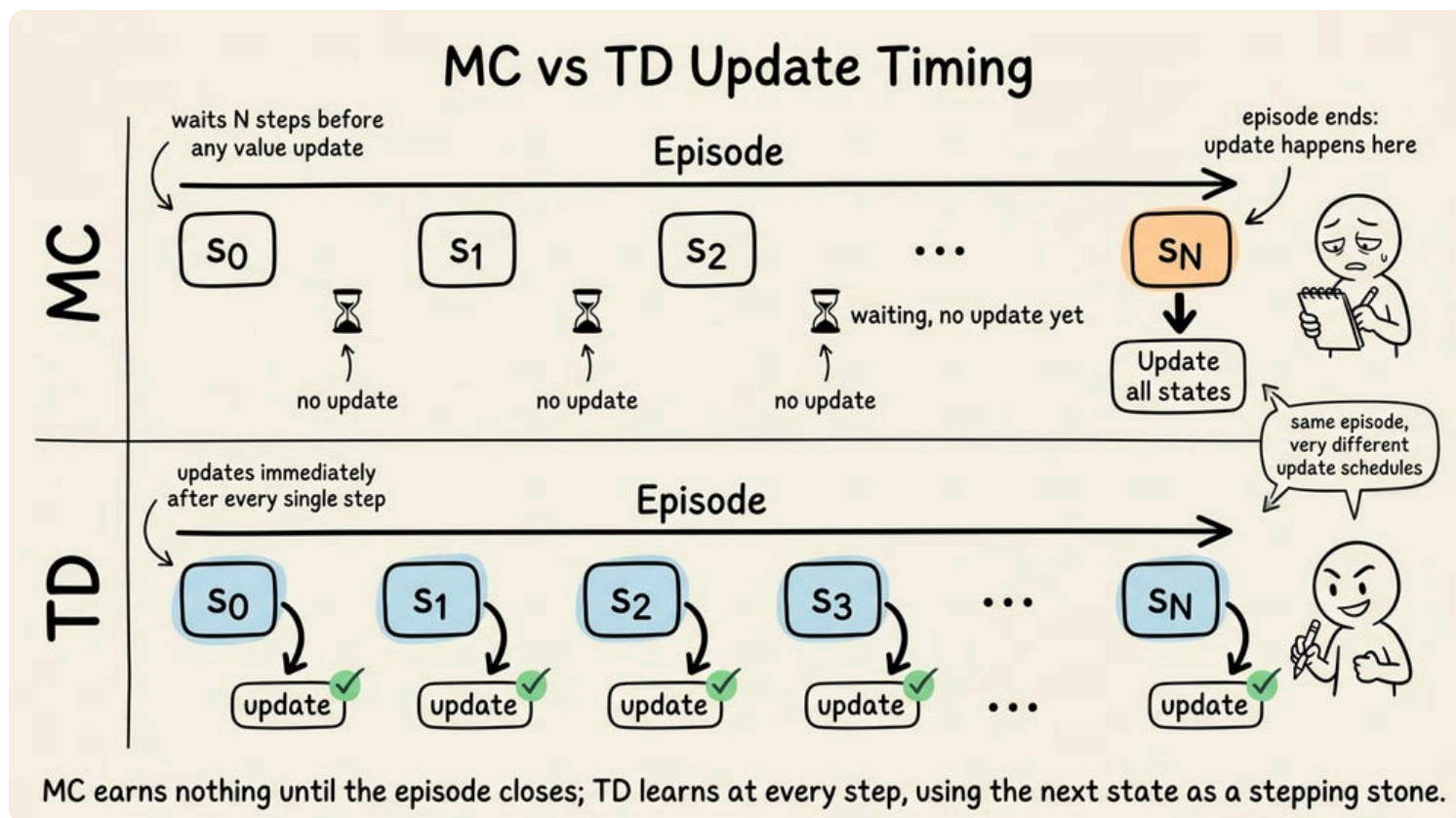
We began by clarifying what "model-free" means. The environment still has dynamics, of course. The point is that the algorithm does not get to see P or R directly. It only gets to interact, observe rewards, and adapt.



We then covered Monte Carlo methods that estimate values by averaging full episode returns. They are unbiased but high variance, and they only work on episodic tasks because the return has to be computable.

Temporal-difference learning sits between MC and DP: it inherits the model-free property from MC and the bootstrapping idea from DP. The TD(0) update uses

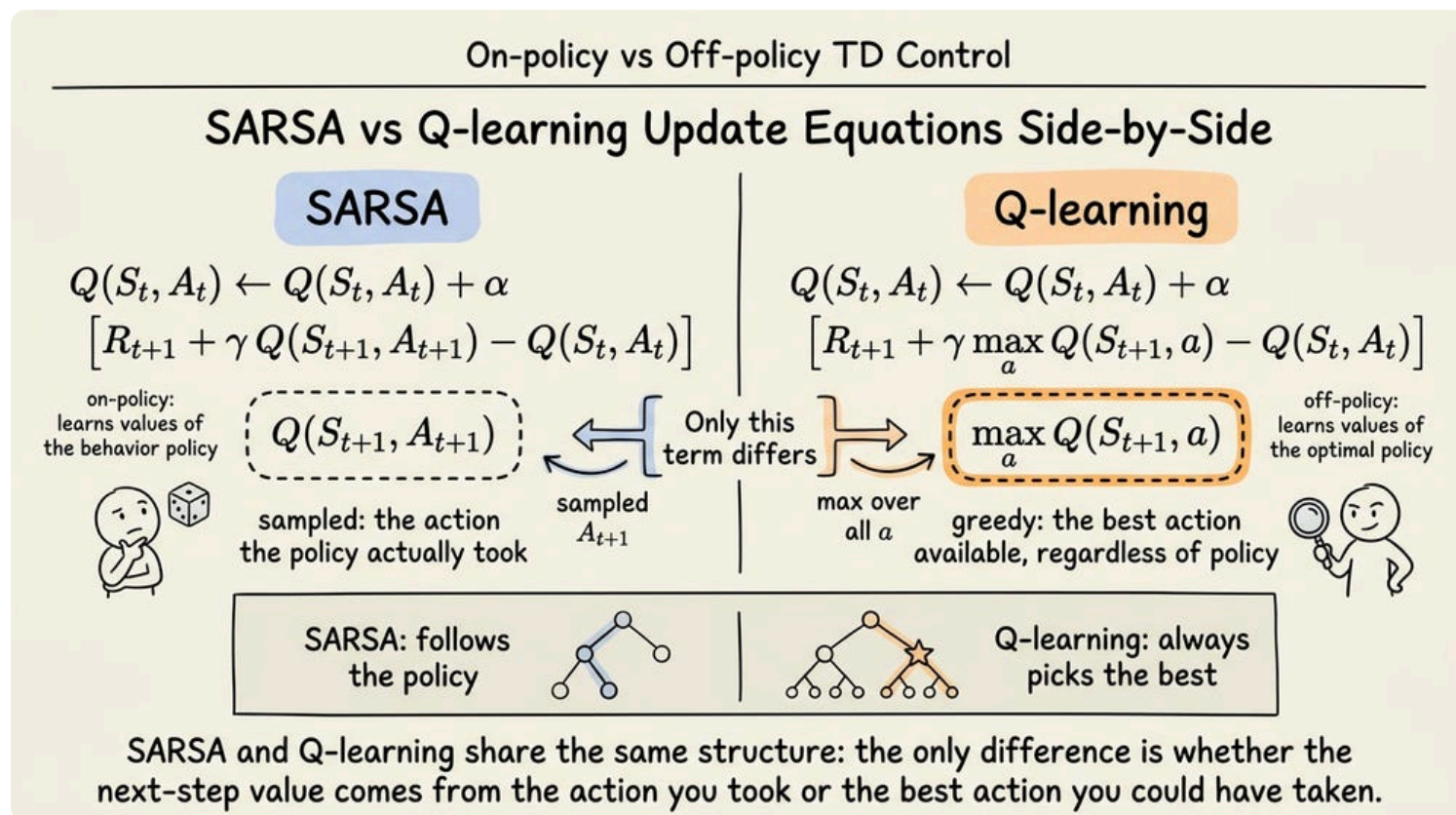
one observed reward plus the current estimate of the next state's value, which lets the agent learn online, after every single step.



From TD prediction, we moved to TD control:

- SARSA is the on-policy variant, using the value of the next action that the policy actually picks.

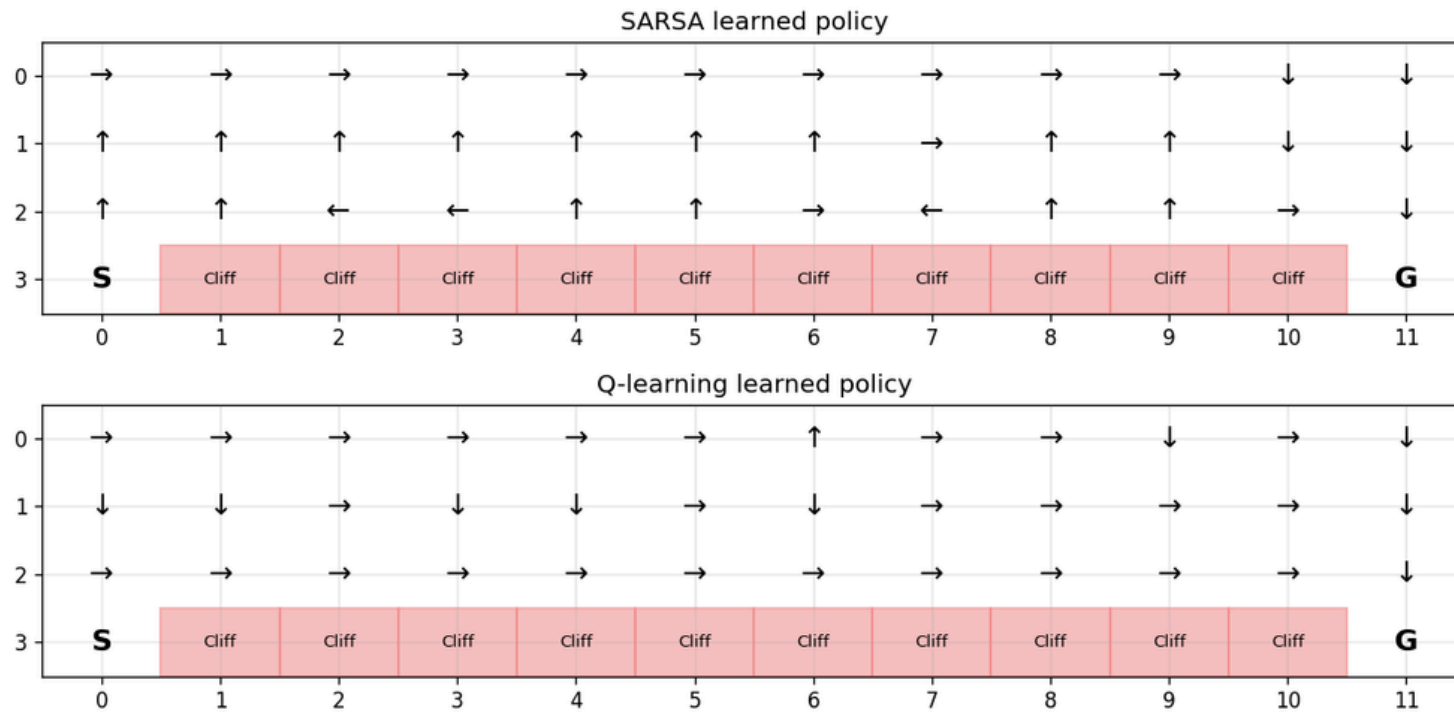
- Q-learning is off-policy, using the max over next actions, so it can learn the optimal greedy policy while the agent itself explores.



We closed with a hands-on experiment on the Cliff Walking gridworld:

- Q-learning found the optimal cliff-edge path but suffered more during training because ϵ -greedy exploration kept knocking it off the cliff.

- SARSA learned the safer path along the top and accumulated less negative reward, because its update reflects the policy that is actually being followed.



If you have not read Chapter 4, we recommend doing so first:

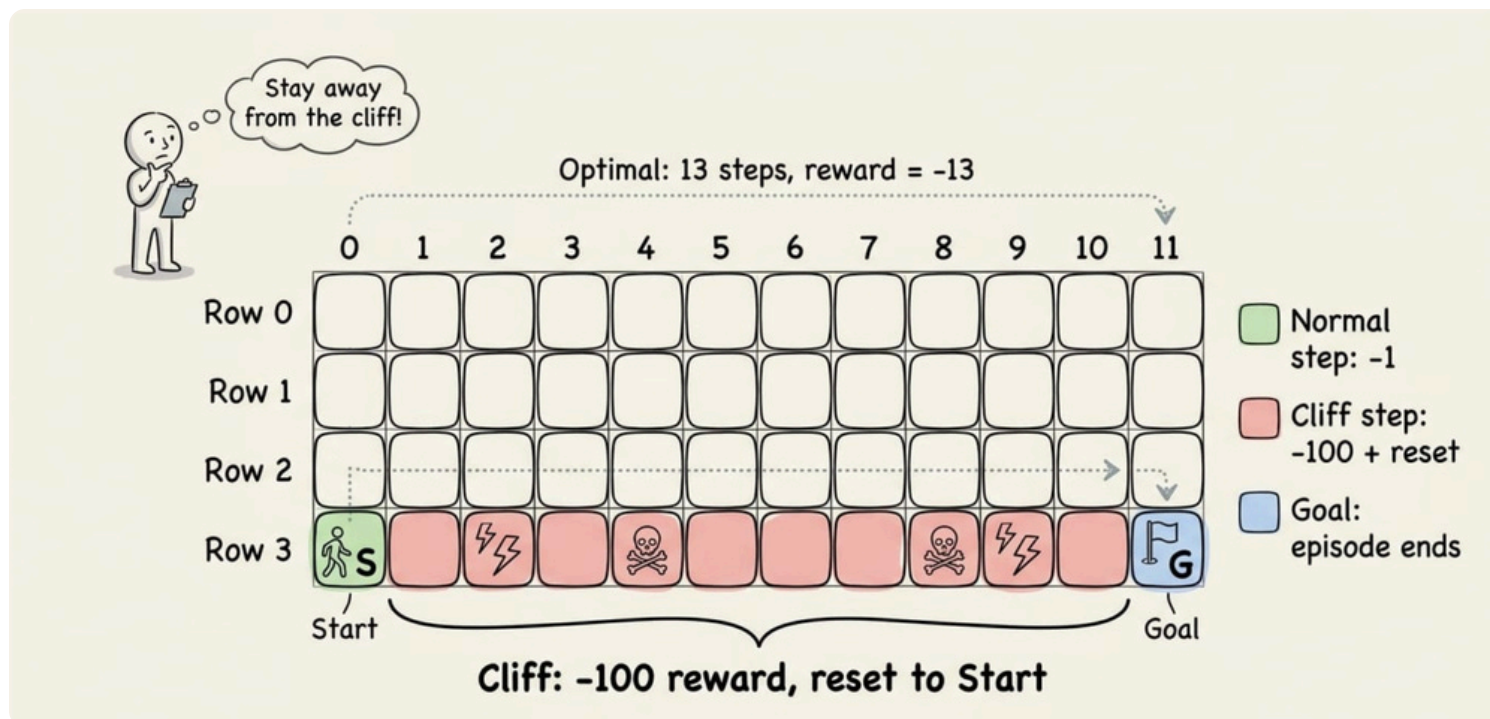
Model-Free Learning

RL Part 4: Learning value functions and policies without a model. Monte Carlo methods, TD(0), SARSA, Q-learning, and the bias-variance bridge between...



Introduction

Everything we did so far fits into tables, like the Q-table in chapter 4 for Cliff Walking had 48 cells.



Now consider something like Backgammon, where the number of distinct positions is on the order of 10^{20} , or mountain car, the classic control benchmark we will meet later in this chapter, where the state is a pair of real numbers (position and velocity) drawn from a continuous range. The table approach has nowhere to go.

This chapter is about knowing that bridge:

- We will replace the table with a parameterized function.
- We will set up a learning objective.
- Derive the gradient Monte Carlo and semi-gradient TD updates.
- Extend the control algorithms to the approximation setting.

We will then learn about the deadly triad, the combination of function approximation, bootstrapping, and off-policy learning that can cause value estimates to diverge.

Finally, we will train a semi-gradient SARSA agent on mountain car using tile coding, the workhorse linear-features technique for continuous state spaces, and plot the learned cost-to-go surface.



This chapter is conceptually dense and assumes strong familiarity with the previous chapters, as well as a solid understanding of probability and linear algebra. It is recommended to follow the explanations with pen and paper at hand.

Let's begin!

Why tables fail at scale?

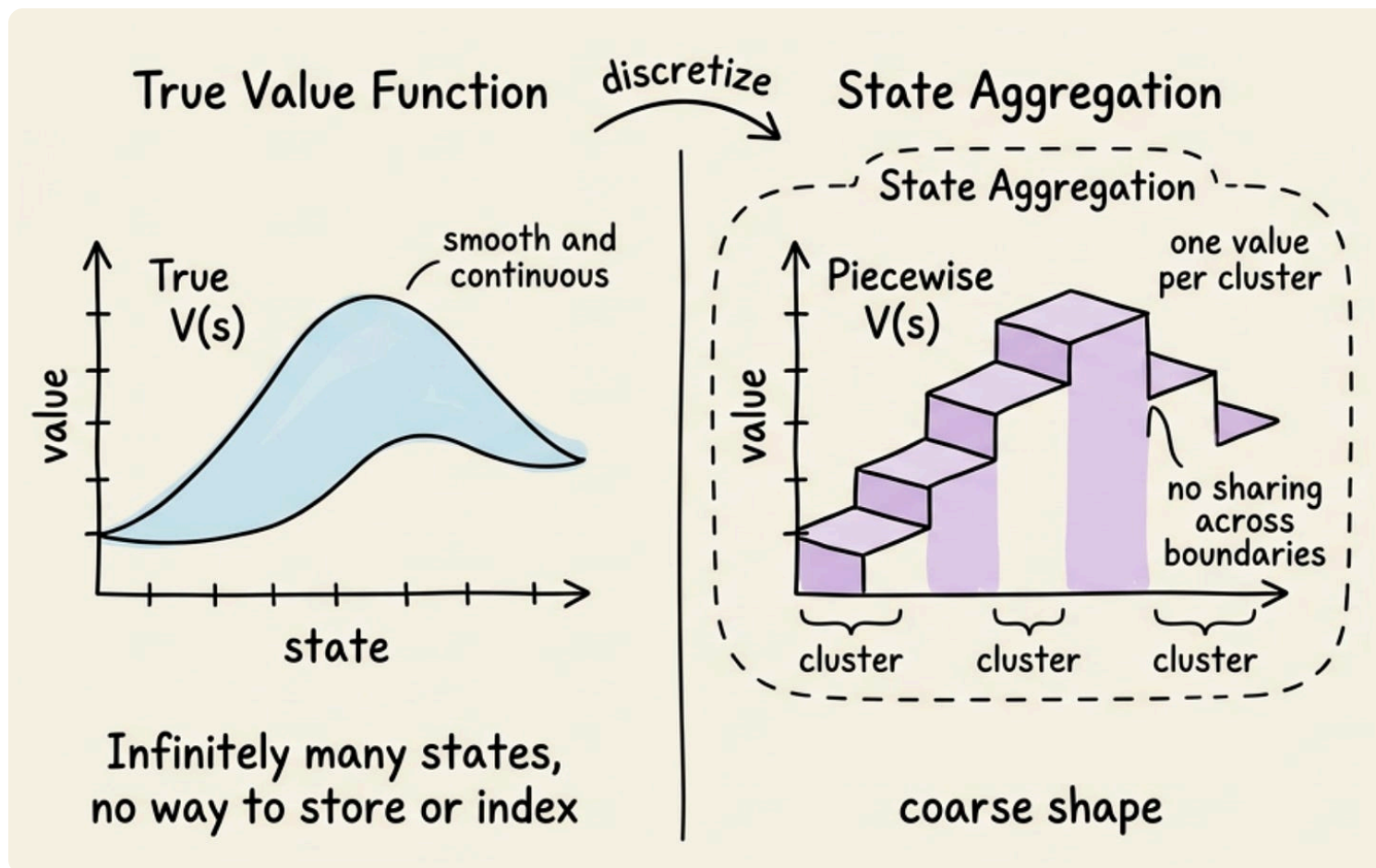
The tabular approach has two failure modes, and they compound:

- **Memory:** When the state space is small and discrete, this is fine. When it is large, you cannot store the table. When it is continuous, you cannot even index into it.
- **Zero generalization:** Updating the entry for state s changes nothing about state s' , even if s' is right next to s . Every state-action pair has to be visited many times for its value to settle. In a space with millions of states, you will never visit enough of them to learn anything useful, even if memory were

free. What we need is a representation where updating the value at s also updates nearby states automatically, where similar states share information. This is what function approximation gives us.

A concrete way to feel the problem is to take a car (known as the mountain car example), where the state is just two floats, say position $\in [-1.2, 0.6]$ and velocity $\in [-0.07, 0.07]$. There are infinitely many states. A table is structurally incapable for this. Thus we need function approximation.

One way to do this is state aggregation. Group the state space into clusters and store one value per cluster. It is the simplest possible form of generalizing approximation. For example, a 1000-state random walk can be grouped into 10 clusters of 100 states each.



The resulting value function is a staircase, which holds a piecewise constant value within each cluster, jumping between clusters. It is crude, but it captures the broad shape of the true value function using only 10 parameters instead of 1000.

In summary, tables are out, not because they are wrong, but because they cannot scale and cannot generalize. The fix is to represent the value function as a function over states, parameterized by a small set of weights.

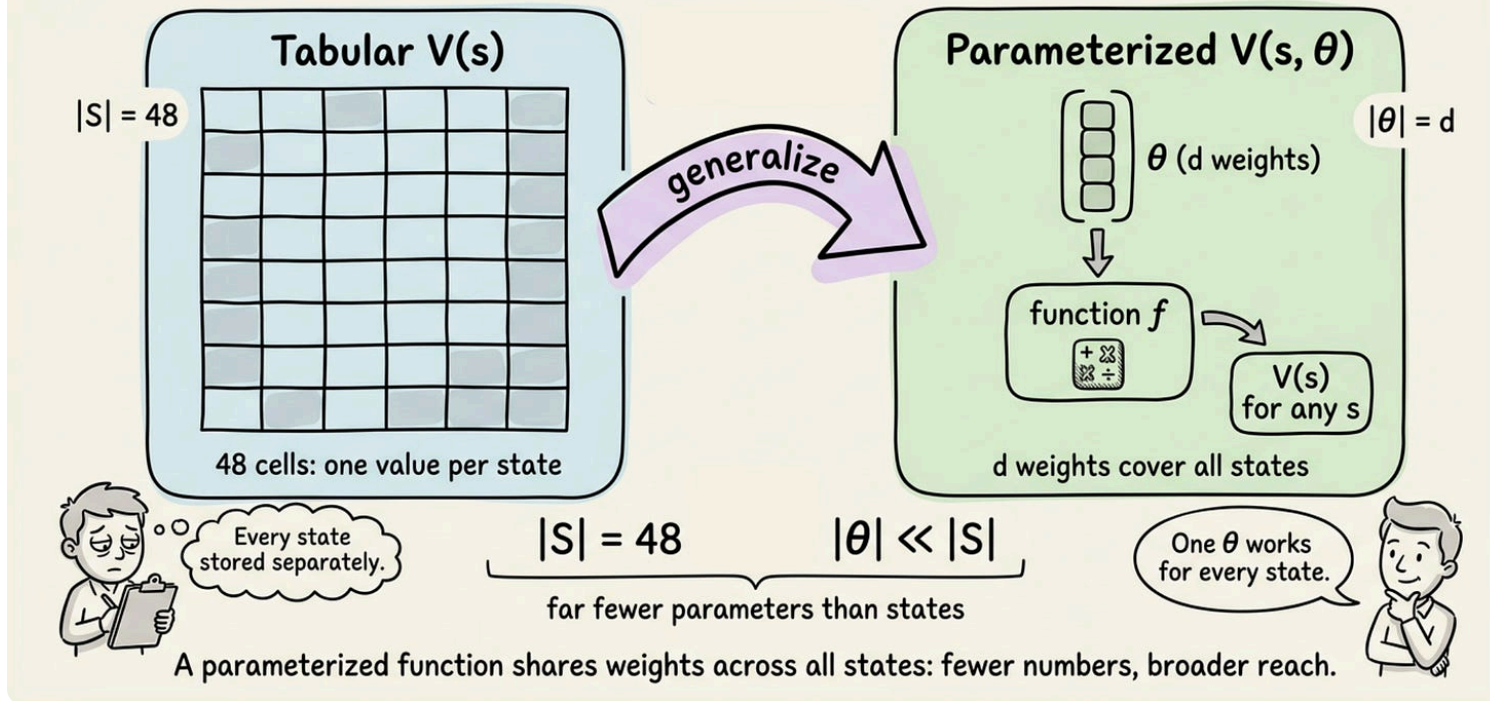
Parameterized value functions

The shift is from a table to a function. We write:

$$\hat{v}(s, \theta) \approx v_{\pi}(s)$$

where $\theta \in \mathbb{R}^d$ is a parameter vector of dimension d , typically much smaller than $|\mathcal{S}|$.

Tabular V to Parameterized V



The function $\hat{v}$ can be anything that is differentiable in θ , for example, a linear combination of features.

The same idea applies to action values:

$$\hat{q}(s, a, \theta) \approx q_{\pi}(s, a)$$

The change is more profound than it looks. With a table, updating $V(s)$ for one state affected only that state. With a parameterized function, changing θ to fit the value of state s also changes the predicted values at every other state, because $\hat{v}$ depends on all the components of θ .

This is the generalization we need. However, it is also the source of all the new difficulties in this chapter. We can no longer make every state's estimate exactly right. The parameters are shared, so making one state more accurate can make another less accurate. Thus, we are obligated to choose what we care about.



The number of parameters d is typically far smaller than $|\mathcal{S}|$. This is not a limitation, it is the design. Sharing parameters is what produces generalization.

The mean square value error

The tabular case did not need an explicit objective. Updates were decoupled, and each estimate could march independently toward its true value. With

function approximation, we lose that luxury, so we have to specify what "good" means.

The standard prediction objective is the mean square value error, or MSVE. It is a weighted average of squared prediction errors across states:

$$\text{MSVE}(\theta) = \sum_{s \in \mathcal{S}} d(s) [v_{\pi}(s) - \hat{v}(s, \theta)]^2$$

Diagram illustrating the MSVE equation with annotations:

- $v_{\pi}(s)$: true value of s under policy π (green text)
- $d(s)$: non-negative weighting (red text)
- $\hat{v}(s, \theta)$: our parameterized estimate at s (blue text)

Let's unpack this:

- $v_{\pi}(s)$ is the true value of state s under policy π
- $\hat{v}(s, \theta)$ is our parameterized estimate at that state
- The squared bracket measures pointwise error at s .
- The factor $d(s)$ is a non-negative weighting that distributes our care across the state space.

- The sum runs over every state.

The structure tells us two things:

- Error at each state is squared, so positive and negative errors do not cancel, meaning every deviation costs us.
- $d(s)$ controls which states matter. A state with $d(s) = 0$ contributes nothing to the objective no matter how badly we mispredict it, and a state with high $d(s)$ dominates.

The intuition is that with limited capacity, we cannot get everything right, so we have to triage. The natural choice for $d(s)$ is the on-policy distribution (the long-run fraction of time the agent spends in each state when following π). The states the policy visits often get prioritized. The states the policy never visits, we cannot learn about them anyway, so weighing them is irrelevant.

A useful sanity check is that if $d(s)$ were a uniform distribution and $\hat{v}$ were perfect everywhere, MSVE would be zero. Any deviation lifts it above zero, weighted by where we care.



MSVE is the standard objective for prediction. It is not provably the right objective for control: the value function that minimizes MSVE may not be the value function that yields the best policy when used greedily.

One important thing to note is that we usually do not know the true value function $v_{\pi}(s)$. Because of that, we cannot compute MSVE directly.

Instead, MSVE serves as a theoretical objective that tells us what good predictions look like. The learning algorithms we build update the parameters in ways that, in expectation, descend MSVE, and we will track how closely they actually do so.

Linear function approximation

The most important special case is when $\hat{v}$ is linear in θ . This is the case where everything generalizes cleanly from the tabular setting and where almost every theoretical result lives.

Each state is represented by a feature vector:

$$\phi(s) = (\phi_1(s), \phi_2(s), \dots, \phi_d(s))^T \in \mathbb{R}^d$$

The value estimate is then the inner product of features and weights:

$$\hat{v}(s, \theta) = \theta^T \phi(s) = \sum_{i=1}^d \theta_i \phi_i(s)$$

The terms:

- $\phi_i(s)$ is the i -th feature evaluated at state s , a fixed function of the state.
- θ_i is the learnable weight for that feature.
- The product $\theta_i \phi_i(s)$ is the contribution of feature i to the predicted value at s , and we add up all such contributions.

The intuition has two parts:

- First, the features carry the inductive bias: similarity in feature space implies similarity in predicted value. Two states with nearly identical feature vectors must have nearly identical predicted values, by construction.
- Second, the weights carry the learning: as θ changes, every state's predicted value changes in a coordinated way determined by the features.

The gradient is the cleanest part. Differentiating $\hat{v}(s, \theta) = \theta^\top \phi(s)$ with respect to θ gives $\phi(s)$. That is, the gradient at state s is just the feature vector at s . This is what makes linear methods so tractable.



The tabular case is a special instance of linear FA. If you choose features to be one-hot indicators (one feature per state, equal to 1 at that state and 0 elsewhere), then θ is the table, $\phi(s)$ selects one entry, and the linear method reduces exactly to the tabular update.

In summary, linear FA gives us a representation with two pieces: a fixed feature map ϕ that carries the inductive bias, and a learnable weight vector θ that

carries the learning. The gradient is the feature vector itself, which makes everything that follows simple to derive and to implement.

Gradient Monte Carlo

Let's now understand our first learning algorithm for the function approximation setting. The idea is to treat each visit to a state as a supervised training example.

The supervised setup is this:

→ input S_t , target G_t (the return observed from S_t onward), and we want to make $\hat{v}(S_t, \theta)$ close to G_t . The standard tool is stochastic gradient descent on the squared error.

After observing (S_t, G_t) , we adjust θ in the direction that most reduces $[G_t - \hat{v}(S_t, \theta)]^2$. That is:

$$\theta \leftarrow \theta + \alpha [G_t - \hat{v}(S_t, \theta)] \nabla_{\theta} \hat{v}(S_t, \theta)$$

The terms:

- G_t is the actual observed return from time t , computed by playing the episode out to termination.
- $\hat{v}(S_t, \theta)$ is the current estimate.
- The bracketed difference is the prediction error.
- The gradient $\nabla_{\theta} \hat{v}(S_t, \theta)$ is the direction in parameter space that increases $\hat{v}(S_t, \theta)$ fastest.
- α is the step size.

For linear function approximation, the squared error is convex in θ , so the local minimum is the global minimum. Gradient Monte Carlo with linear FA provably converges to the parameter vector that minimizes MSVE, under the on-policy distribution.

Now, coming to limitations, they are also the same as in the tabular case:

- Gradient MC is episodic-only, since G_t has to be computable.
- It has to wait for the episode to end before any update happens.
- It inherits MC's high variance: the return is a sum of many rewards, so two samples from the same state can differ enormously.



Gradient MC is the only algorithm in this chapter that is a true gradient method on a well-defined squared-error objective. Every other algorithm we look at relaxes this in some way and pays a theoretical price for it.

These three problems are exactly what motivated TD in the tabular setting. Let's apply the same fix here.

Semi-gradient TD(0)

Suppose we want to replace the MC target G_t with a TD-style one-step target.
The natural choice is:

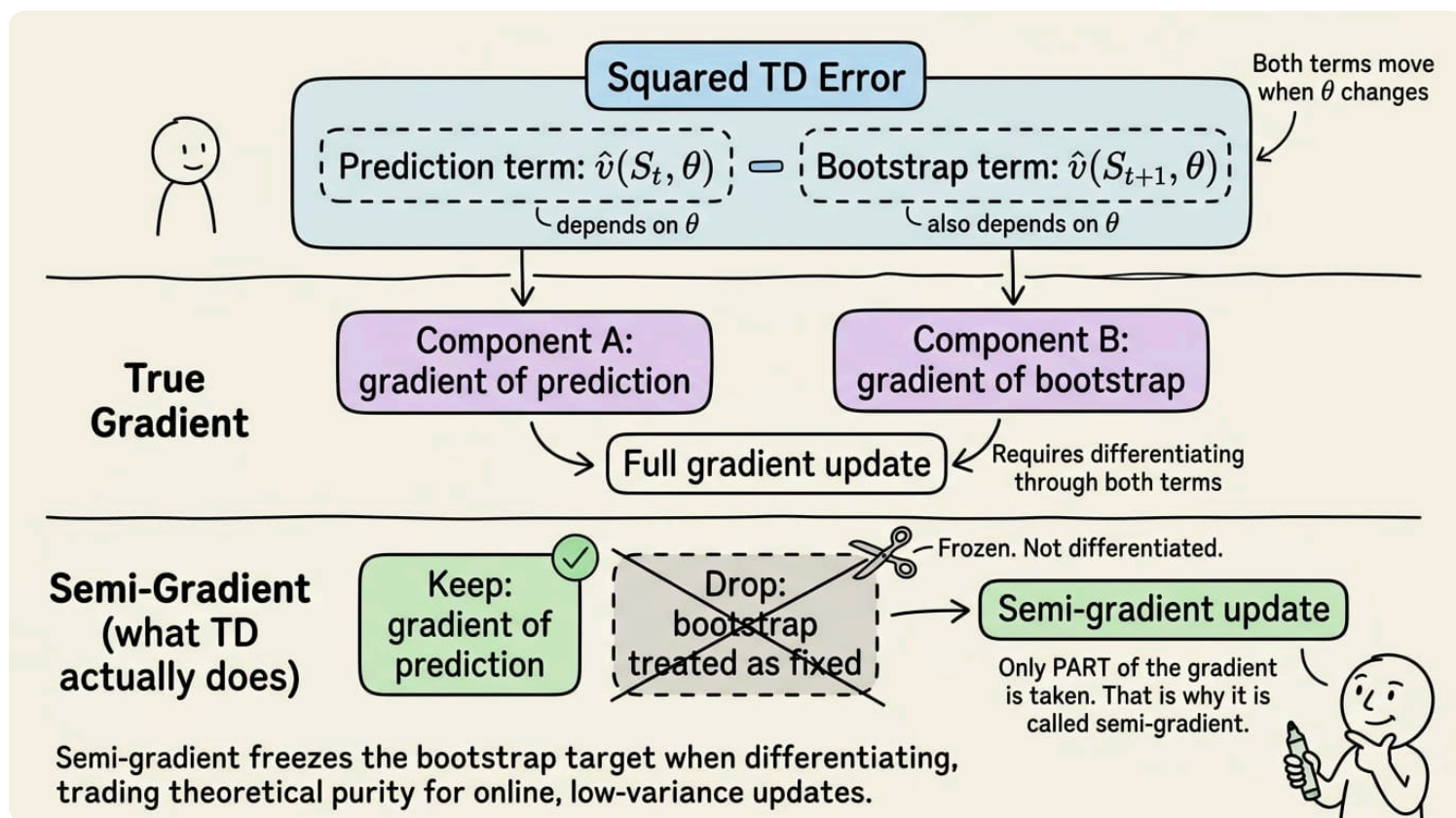
$$U_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, \theta)$$

This uses one observed reward, then bootstraps off the current value estimate of the next state. The update becomes:

$$\theta \leftarrow \theta + \alpha [R_{t+1} + \gamma \hat{v}(S_{t+1}, \theta) - \hat{v}(S_t, \theta)] \nabla_{\theta} \hat{v}(S_t, \theta)$$


The target, $R_{t+1} + \gamma \hat{v}(S_{t+1}, \theta)$, depends on θ . If we took a true gradient of the squared TD error, we would have to differentiate through both $\hat{v}(S_t, \theta)$ (the prediction) and $\hat{v}(S_{t+1}, \theta)$ (the bootstrapped target).

But we do not; we deliberately freeze the target and only differentiate the prediction. That is what "semi" in semi-gradient means: only part of the gradient is taken. The bootstrap term is treated as if it were a fixed target, when in reality it moves whenever θ moves.



This sounds counterintuitive, and in a sense it is. The theoretical convergence guarantees we had for gradient MC do not carry over. The update is not a true

gradient of any objective, so we cannot lean on stochastic gradient descent convergence theorems.

So why do we use it? There are two reasons for this:

- Bootstrapped targets have far lower variance than full Monte Carlo returns.
- Semi-gradient methods can update online, after every single step, on continuing tasks.

The same advantages we had with tabular TD over tabular MC carry over here too.

And in important settings, semi-gradient TD does converge. In particular, linear semi-gradient TD(0) with on-policy sampling is guaranteed to converge to a unique stable solution called the TD fixed point.

$$\theta_{TD} = A^{-1}b$$

The diagram illustrates the components of the TD fixed point formula. A green arrow points from the expression $\mathbb{E}[\phi_t(\phi_t - \gamma\phi_{t+1})^\top]$ to the A^{-1} term in the formula $\theta_{TD} = A^{-1}b$. A blue arrow points from the expression $\mathbb{E}[R_{t+1}\phi_t]$ to the b term.

We will not derive this full formula here; the intuition matters more. The TD fixed point is simply the parameter vector where the expected TD update becomes zero. In other words, once learning reaches this point, the updates stop changing the parameters on average.

However, this solution is generally not the one that minimizes MSVE. The solution found by gradient Monte Carlo usually achieves a lower prediction error because it directly optimizes MSVE, whereas TD learning relies on bootstrapping.

Still, the important result is that TD's solution is provably well-behaved. Tsitsiklis and Van Roy (1997) showed that the error at the TD fixed point cannot be arbitrarily worse than the error of the best possible linear approximation. So

even though TD may not find the optimal approximation, it still converges to a stable and useful one.



Two important caveats sit behind this result. First, it assumes linear FA. With nonlinear FA, even on-policy semi-gradient TD can diverge. Second, it assumes on-policy sampling. With off-policy sampling, divergence is possible even with linear FA. We will see this happen in the Baird's counterexample experiment.

In summary, semi-gradient TD trades a clean convergence guarantee for online, low-variance updates that work on continuing tasks. In the on-policy linear case, the trade is favorable: we lose the MSVE optimum but gain everything else.

Semi-gradient SARSA and Q-learning for control

The shift from prediction to control follows exactly the pattern we used in Chapter 4: replace state values with action values, and act ε -greedily with respect to them.

We now have a parameterized action-value function $\hat{q}(s, a, \theta)$, with features $\phi(s, a)$ that depend on both state and action. There are several ways to construct such features. One common choice is to compute state features $\phi(s)$ and "stack" them across actions, so each action gets its own copy of the state-feature weights. We will use this idea in the Mountain Car hands-on through tile coding's integer-tagging mechanism.

The semi-gradient SARSA update is:

$$\theta \leftarrow \theta + \alpha [R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}, \theta) - \hat{q}(S_t, A_t, \theta)] \nabla_{\theta} \hat{q}(S_t, A_t, \theta)$$

This mirrors tabular SARSA exactly. The TD target uses A_{t+1} , the action that the current policy actually picks at S_{t+1} . The update is on-policy: the policy that generates behavior is the same policy whose value we are estimating.

The semi-gradient Q-learning update changes one thing:

$$\theta \leftarrow \theta + \alpha [R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \theta) - \hat{q}(S_t, A_t, \theta)] \nabla_{\theta} \hat{q}(S_t, A_t, \theta)$$

The max over a replaces the sampled A_{t+1} , making the update off-policy. The target is the value of the greedy action at the next state, regardless of what action the behavior policy actually takes.

Now let's talk about convergence. For semi-gradient SARSA with linear FA and a fixed ε -greedy policy, the situation is similar to TD prediction. The algorithm tends to chatter, oscillating in a bounded region rather than converging to a single parameter vector, because the policy keeps changing as $\hat{q}$ updates. In practice, it works well on tasks like Mountain Car.

For semi-gradient Q-learning with linear FA, the off-policy nature plus the max operator plus the bootstrapping is the dangerous combination we are about to see fail. There is no general convergence guarantee. Sometimes it works; sometimes the weights diverge.

In summary, control with function approximation reuses the SARSA and Q-learning ideas from Chapter 4, but the convergence picture gets harder. Semi-

gradient SARSA is reasonably well-behaved on-policy. Semi-gradient Q-learning is off-policy and lives one step closer to the deadly triad.

The deadly triad

Three properties are individually desirable in a reinforcement learning algorithm:

- Function approximation, because tables do not scale.
- Bootstrapping, because waiting for full returns is slow and high-variance.
- Off-policy learning, because we want to learn about an optimal policy while exploring with a different one (this is what makes Q-learning attractive).

The trouble was identified by Baird (1995) and Tsitsiklis and Van Roy (1997), and was named the "deadly triad" by Sutton and Barto. When all three properties are used together, value estimates can diverge. The parameters do not just fail to converge; they can grow without bound.

Why is the triad dangerous?

The short answer is that each ingredient introduces a kind of amplification, and the three amplifications compound.

- Function approximation generalizes: changing θ to fit one state changes the predicted values of many other states.
- Bootstrapping means the update target depends on those predicted values.
- Off-policy learning means the distribution over which we sample updates does not match the distribution that the bootstrapped targets are implicitly assuming.

The combination can set up a feedback loop. A prediction error at one state propagates through the function approximator to states it shouldn't. It then gets fed back into the bootstrapped target, and grows on each update.

However, it is worth being precise about what is and is not at risk. Any two of the three ingredients without the third is provably fine. For example:

- Tabular methods with bootstrapping and off-policy (no FA): tabular Q-learning converges.
- Function approximation with bootstrapping and on-policy: linear semi-gradient TD on-policy converges.

The triad is specifically about combining all three.



The phrase "deadly triad" can sound alarming. The reality is that all three ingredients are useful in isolation. Each pair is also fine. It is only the specific three-way combination that creates the divergence risk, and that risk is realized only on certain problems. The deadly triad describes a possibility of divergence, not an inevitability.

The deadly triad is the central practical concern in modern value-based deep RL. DQN is essentially Q-learning with neural network function approximation, which sits squarely in the danger zone. The fact that DQN works at all is due to two specific engineering choices, target networks and experience replay, both of which exist primarily to tame the triad. We will see this in detail in a future chapter.

For now, we have understood all the key concepts for this chapter. The pieces are in place. So let's go ahead and dive into hands-on section for some practical experimentation.

Hands-on demonstrations

Baird's counterexample

Baird's counterexample is the standard textbook demonstration that the deadly triad can produce divergence. It is a tiny linear system with no neural networks, no exploration noise, and not much going on at all. The divergence it produces is monotonic and unambiguous, which strips the phenomenon down to its bones.

The code and project setup are attached below as a zip file. You can extract it and run `uv sync` to get going.

```
.python-version  
baird.py  
pyproject.toml  
uv.lock
```

Download the zip file below:

baird-counterexample

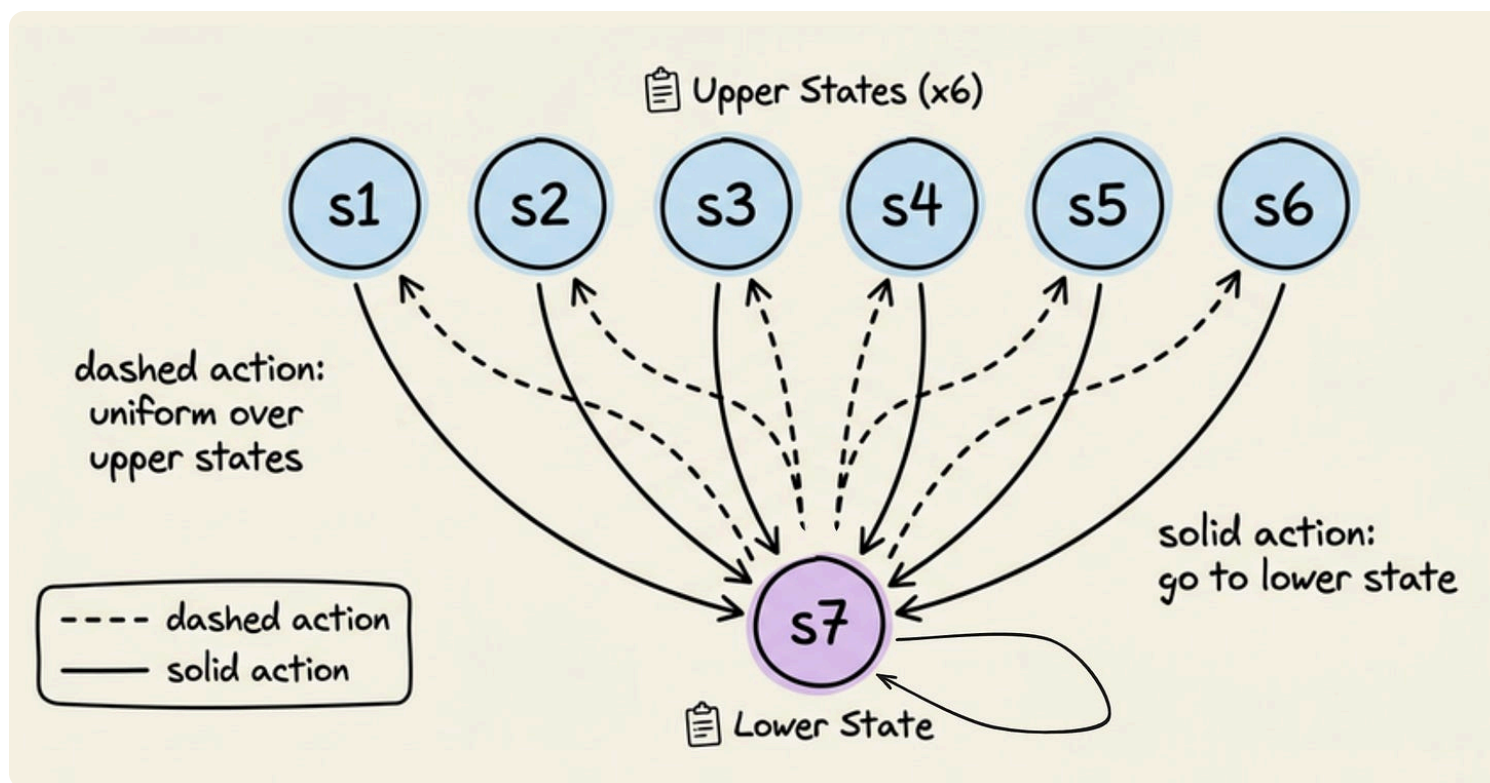
baird-counterexample.zip • 36 KB



For details about versions and dependencies, check the `.python-version` and `pyproject.toml` files.

The environment

The MDP has seven states. Six are "upper" states, one is a "lower" state. There are two actions, traditionally called `dashed` and `solid`. The dashed action sends the agent to one of the six upper states uniformly at random. The solid action sends the agent deterministically to the lower state.



All rewards are zero. The discount factor is $\gamma = 0.99$.

The behavior policy b picks the dashed action with probability $6/7$ and the solid action with probability $1/7$. The target policy π picks the solid action with probability 1, deterministically.

Because all rewards are zero and the episode is non-terminating in any meaningful sense, the true value function under any policy is $v_\pi(s) = 0$ for every state s .

The features are designed by Baird specifically to expose the instability. There are seven states and eight weights $w_1, \dots, w_8$. The feature vector for upper state i (for $i = 1, \dots, 6$) puts a 2 in the i -th slot and a 1 in slot 8. The lower state puts a 1 in slot 7 and a 2 in slot 8. So state i has value $\hat{v}(s_i, \theta) = 2w_i + w_8$ for the upper states, and $\hat{v}(s_7, \theta) = w_7 + 2w_8$ for the lower state.

The true value function $v_\pi = 0$ is representable: just set $\theta = 0$ and every $\hat{v}$ is zero. The features have enough capacity, so this is not a representational failure. The failure will be entirely in the learning dynamics.

Implementation

Let's now dive into the code:

Here:

- `build_feature_matrix()` constructs the feature matrix $\Phi \in \mathbb{R}^{7 \times 8}$, where each row corresponds to the feature vector $\phi(s)$ for one of the seven states. This matrix defines the linear function approximator used in Baird's counterexample and encodes the intentionally pathological feature structure that causes divergence.
- `run_expected_semi_gradient_dp()` implements the deterministic expected-update version of semi-gradient TD. Instead of sampling transitions from interaction with the environment, the algorithm directly applies the expected TD update across all states at every sweep. Under the target policy, every state transitions to state 7 (state 6, by index), so the update uses $\hat{v}(s_7, \theta)$ as the bootstrap target for every state.
- For each state, the TD error is $0 + \gamma \hat{v}(s_7, \theta) - \hat{v}(s, \theta)$, since all rewards are zero. The semi-gradient update multiplies this TD error by the feature

vector $\phi(s)$ and accumulates the result across states before updating the weights.



Because this version uses the exact expected update operator, there is no sampling noise. The key point is that the weights still diverge. This shows that the instability is inherent to the update rule itself, not caused by randomness in the samples.

Moving ahead:

The code also includes `run_semi_gradient_off_policy_td()`, which implements the sampled off-policy TD version. Here, transitions are generated using the behavior policy, and updates are corrected using the importance sampling ratio:

$$\rho = \frac{\pi(A_t | S_t)}{b(A_t | S_t)}$$

The behavior policy chooses the dashed action with probability $6/7$ and the solid action with probability $1/7$, while the target policy always chooses the solid action.

As a result:

- for the dashed action, $\rho = 0$ because the target policy would never take that action, so those updates contribute nothing,
- for the solid action, $\rho = 7$.



The importance sampling ratio ρ corrects for the mismatch between the behavior policy and the target policy. In the TD update, ρ simply scales the update size. If the behavior policy takes the dashed action, then $\rho = 0$ because the target policy would never take that action, so the update is ignored. If the behavior policy takes the solid action, then $\rho = 7$ because the target policy always takes that action while the behavior policy takes it only with probability $1/7$. As a result, those updates are amplified to compensate for the fact that they are sampled rarely under the behavior policy.

This sampled version more closely resembles how a real RL agent would learn from experience, but its updates are noisy because they depend on randomly sampled states and actions.

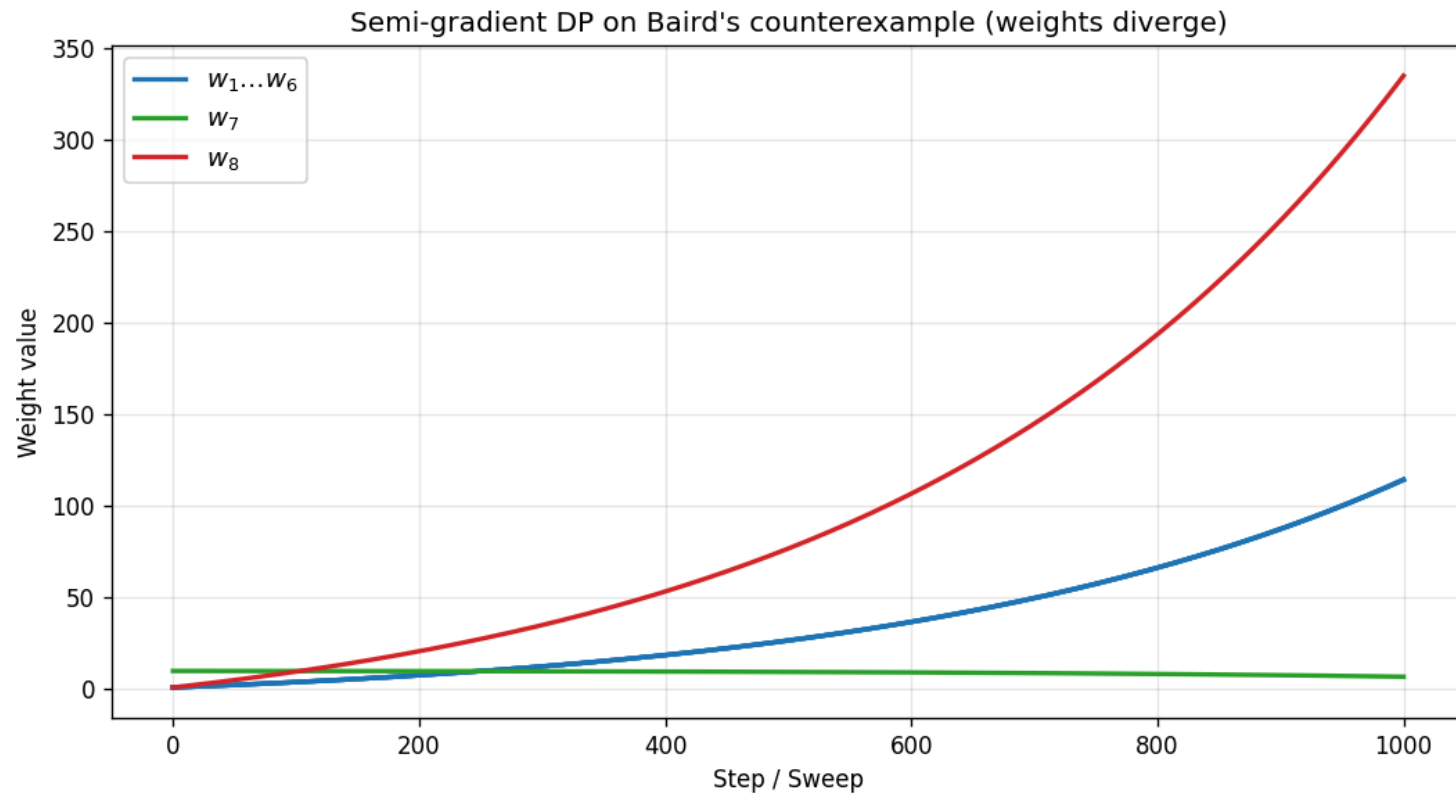
Results

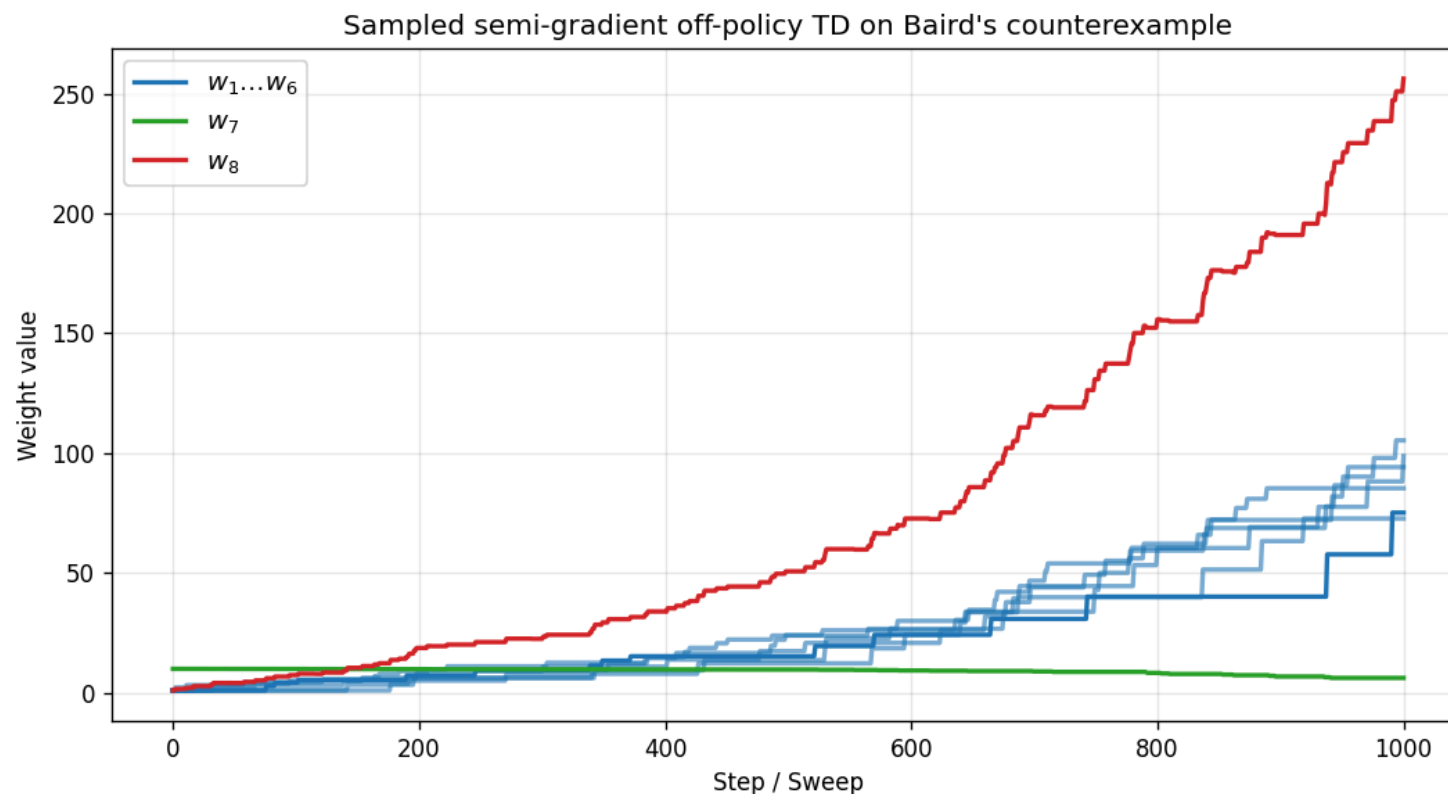
Running the code produces:

```
Running expected semi-gradient DP on Baird's counterexample...
Final weight magnitudes (DP): [114.43371614 114.43371614 114.43371614 114.43371614 114.43371614
| 114.43371614 6.85003034 335.00120911]
Max |weight| at end (DP): 3.350e+02

Running sampled semi-gradient off-policy TD on Baird's counterexample...
Final weight magnitudes (TD): [ 75.18772409 98.79943911 94.19480052 105.35951402 85.35541737
| 72.73480691 6.22115952 256.25817006]
Max |weight| at end (TD): 2.563e+02
Saved plot: baird_dp.png
Saved plot: baird_td.png
```

And the two graphs look like:





The diagnosis is very clean. The true value function is zero everywhere, and the function approximator is fully capable of representing it; setting $\theta = 0$ gives the correct solution exactly. The features themselves are not the problem, and since every reward is zero, there is no misleading reward signal either. In principle, the learning problem should be trivial. Yet the weights still grow without bound.

The reason is that the off-policy semi-gradient TD update becomes unstable. The estimate at the lower state s_7 gets bootstrapped into the upper states through the shared feature weight w_8 . Because many states share this component, an increase in the estimate at one place feeds back into updates elsewhere. Instead of correcting the estimates, the updates reinforce each other.

The off-policy weighting makes this even worse. The importance sampling correction amplifies the updates associated with the target-policy action, so the bootstrap signal keeps getting pushed back into the system with increasing strength. Rather than damping errors over time, the update rule creates a positive feedback loop, causing the weights to grow geometrically even though the correct value function is simply zero everywhere.



The takeaway is not that TD is broken. The takeaway is that the combination of FA, bootstrapping, and off-policy distribution can break it.

Now let's go ahead and take a look at our second hands-on demo.

Mountain car with semi-gradient SARSA

A control problem with a continuous state space, where function approximation actually solves something useful.

The code and project setup are attached below as a zip file. You can extract it and run `uv sync` to get going.

```
.python-version  
agent.py  
pyproject.toml  
tiles3.py  
train.py  
uv.lock
```

Download the zip file below:

mountain-car

mountain-car.zip • 40 KB



For details about versions and dependencies, check the `.python-version` and `pyproject.toml` files.

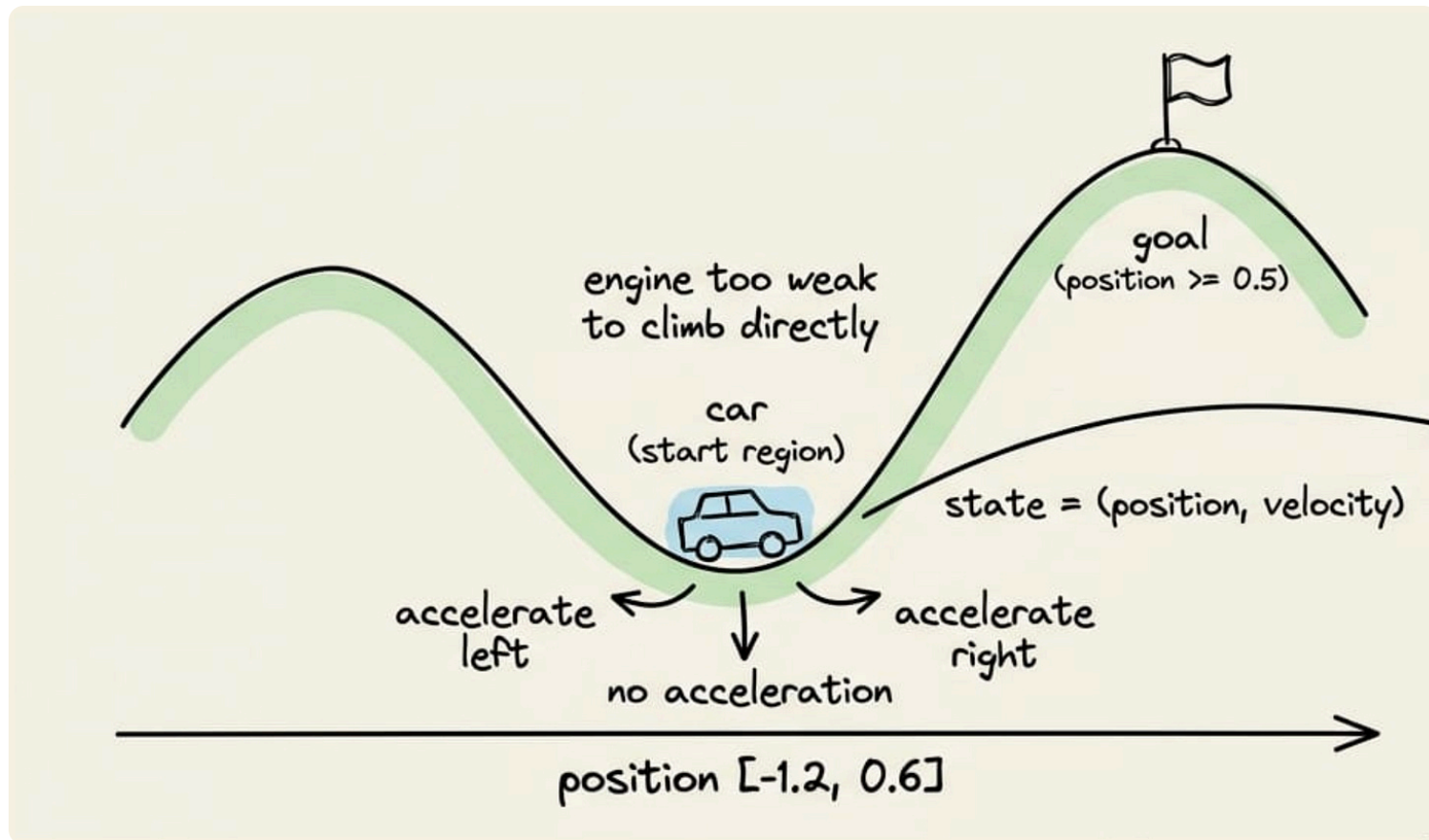
The environment

Mountain car is a classic underactuated control task. An underpowered car sits in a valley between two hills. The goal is to drive the car up the right hill, but the engine is not strong enough to climb directly. The car must build momentum by swinging back and forth.

The state is two-dimensional:

- position $\in [-1.2, 0.6]$
- velocity $\in [-0.07, 0.07]$

The action space has three discrete actions: accelerate left, no acceleration, accelerate right. The reward is -1 at every step until the agent reaches the goal (position ≥ 0.5), at which point the episode terminates.



We use the standard Gymnasium implementation, `MountainCar-v0`, with `max_episode_steps` set to 10,000.

Tile coding

The features we use are based on tile coding, one of the standard linear function approximation methods for continuous, low-dimensional state spaces.

The core idea is simple: divide the state space into a grid of regions called tiles. For any given state, we activate the feature corresponding to the tile containing that state and set all other features to zero. In this sense, a single tiling behaves much like state aggregation or one-hot encoding.

The real power comes from using multiple overlapping tilings instead of just one. Each tiling is shifted slightly relative to the others by a fraction of a tile width. As a result, a state activates one tile in every tiling, so multiple features become active simultaneously.

Nearby states end up sharing many of the same active tiles, which naturally introduces generalization: similar states produce similar feature representations.



The overlapping offsets also provide much finer resolution than a single coarse grid could achieve on its own. A single tiling would create abrupt boundaries, but multiple shifted tilings smooth those boundaries while still preserving locality.

We use Richard Sutton's canonical tile coding implementation, `tiles3.py`, which performs grid-style tile coding with index hashing.

RLAI open web page



In our setup, we use 8 tilings, with each tiling containing 8 tiles per dimension.



Recommended rule of thumb: the number of tilings should be a power of 2 and at least four times the number of float dimensions. We have two float dimensions (position, velocity), so 8 tilings is the minimum acceptable. The number of tiles per dimension controls how fine the partition is within each tiling.

Implementation

Let's now dive into the code:

Here:

- The `TileCoder` class scales position and velocity to the unit range expected by `tiles()` and queries the IHT (index hash table) for the active tile indices for

a given (state, action) pair.

- The `SemiGradientSarsa` agent holds a weight vector θ of length 4096. The action-value $\hat{q}(s, a, \theta)$ is the sum of weights at the active tile indices, since the active features are all 1 and inactive ones are 0. Computing $\hat{q}$ is therefore an indexed sum, not a full dot product.



In mountain car, we use tile coding to turn the 2 continuous variables (position, velocity) into features. With a typical setup of 8 tilings of 8×8 tiles across 3 actions, you theoretically get up to 1536 distinct active features. Tile coding stores these in a hash table, and the table size should be a power of 2 (for fast hashing) and bigger than the number of tiles you'll ever need (to avoid two different states accidentally sharing the same weight). 4096 is just the next power of 2 that comfortably fits, enough room to avoid collisions, small enough to stay cheap.

- Action selection is ε -greedy with random tie-breaking. The update applies the semi-gradient SARSA rule, that is, compute the TD error against the next (state, action) pair, then update the weights at the currently-active tiles (the division by `num_tilings` is the standard step-size scaling for tile coding, since each update touches `num_tilings` active features at once).

Moving ahead to the training loop:

The training script runs semi-gradient SARSA on Mountain Car for 500 episodes. At the start of each episode, the environment resets the car to a random position near the bottom of the valley with zero velocity. The agent then chooses an action, observes the next state and reward, and immediately updates its action-value estimate. This means learning happens online, one transition at a time, rather than after the full episode is over.

During training, we also save snapshots of the learned cost-to-go function at episodes 1, 12, 104, and 500. The cost-to-go is plotted as:

$$- \max_a \hat{q}(s, a)$$

This gives an estimate of how many negative-reward steps remain before reaching the goal from each state. These snapshots let us see how the agent's value function changes as learning progresses. The selected episode numbers are inspired by Sutton and Barto's mountain car example, where they visualize the cost-to-go surface at different stages of learning.

One important detail is that the weights are initialized to zero. In mountain car, every step gives a reward of -1, so successful value estimates become negative. Because of that, an initial value of zero is optimistic: an untried action looks better than actions the agent has already tried and found costly.

This optimism creates natural exploration. Even though the agent uses $\varepsilon = 0$, meaning it always acts greedily, it still explores because unknown actions initially look more attractive than known bad ones. As the agent tries actions and updates their values downward, it gradually discovers better ways to build momentum and reach the goal.

Running the experiment

We use these hyperparameters: $\alpha = 0.1$ (scaled internally to $0.1/8 = 0.0125$ per tile), $\varepsilon = 0$, $\gamma = 1.0$, 8 tilings, 8 tiles per dimension, IHT size 4096, 500 episodes.

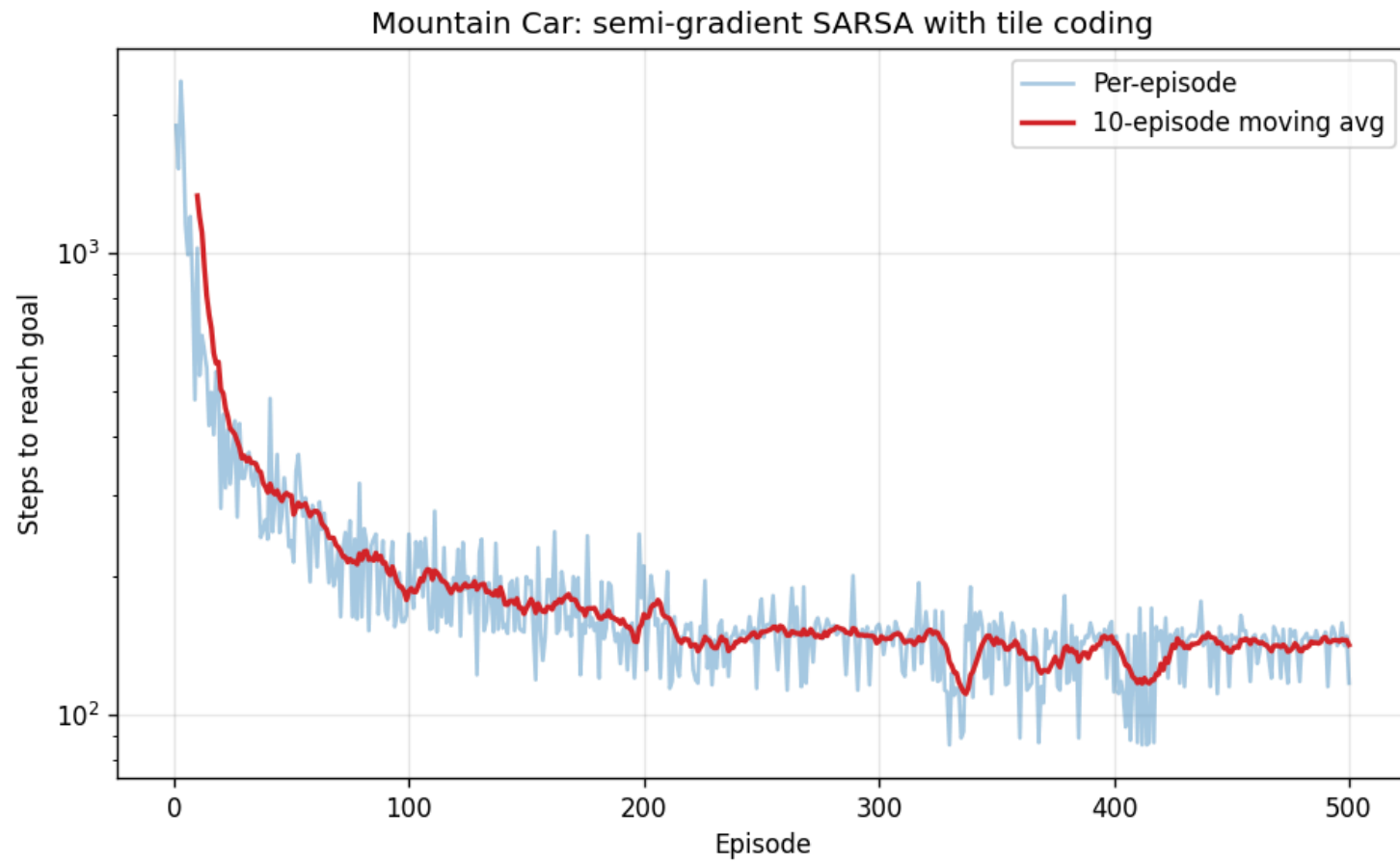
Running `train.py` produces:

```
Training semi-gradient SARSA on Mountain Car...  
Episode 1: steps=1890  
Episode 12: steps=663  
Episode 104: steps=209  
Episode 500: steps=117  
  
Final 20-episode average steps: 144.3  
Best episode steps: 86  
Worst episode steps: 2356  
Saved learning curve: mountain_car_learning_curve.png  
Saved cost-to-go surface: mountain_car_cost_to_go.png
```

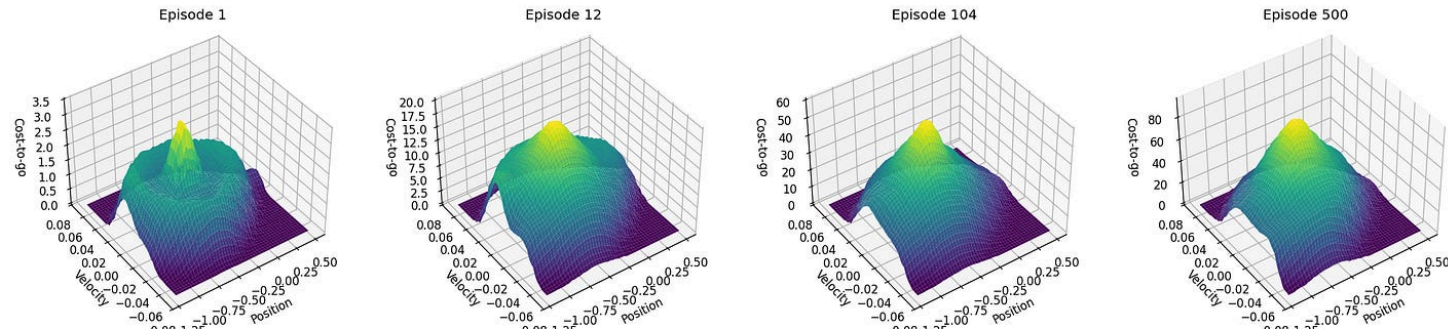
The first episode takes 1,890 steps because the agent is wandering with zero-initialized values. By episode 12 the agent has discovered the back-and-forth strategy and is reaching the goal in roughly 660 steps.

By episode 104, it has converged to a reasonable policy taking around 209 steps. By episode 500, it is at 117 steps per episode. And a best run of 86 steps.

The graph also conveys a similar story:



Now, coming to the cost-to-go surface:



The cost-to-go surface tells the learning story much more clearly.

At the beginning, the surface is noisy and poorly shaped because the agent has not yet learned which states are useful and which are dangerous. As training progresses, the structure becomes smoother and more meaningful.

The highest region of the surface corresponds to states with the largest expected remaining cost to reach the goal. In mountain car, this happens deep in the valley with near-zero velocity. From there, the car has very little momentum and still needs many future steps before it can climb the hill and reach the target.

The lowest region appears near the goal state, where only a few steps remain.



Lower cost-to-go means the agent expects to finish sooner.

Over time, the learned surface develops a smooth gradient across the state space. That gradient reflects how the agent evaluates progress: states with more useful momentum and position receive lower expected cost, while states that are hard to recover from receive higher cost.

Under the greedy policy, the agent chooses actions that move it downhill on this surface toward states with lower expected future cost.

That wraps up the hands-on section. With this, we conclude the discussion for this chapter. In the upcoming chapters, we will continue to build on the core ideas of RL and explore concepts and their implementations wherever applicable.

Conclusion

In this chapter, we made the transition from tables to parameterized value functions. The reasons were structural: tables do not scale and do not generalize. The fix is to represent the value function as a function over states, with a small parameter vector θ controlling its shape.

We laid out the prediction objective, mean square value error, with the on-policy distribution as the natural weighting. We also worked through linear function approximation in detail.

From there, we built two update algorithms:

- Gradient Monte Carlo uses the full return as a target, and because the target is independent of θ , the update is a true gradient method with standard convergence guarantees.
- Semi-gradient TD(0) replaces the return with a bootstrapped target, gaining online updates and low variance, but introducing the bias of "differentiating only the prediction, not the target."

We then extended both ideas to control. Semi-gradient SARSA learns $\hat{q}$ on-policy and works reliably with linear FA. Semi-gradient Q-learning uses the max over actions, making it off-policy and putting it inside the deadly triad.

We also saw this divergence in code. Baird's counterexample produced monotonic, unbounded weight growth despite a true value function of zero and features capable of representing it exactly.

Then Mountain Car showed the other side of the picture: semi-gradient SARSA with tile coding, solving a continuous-state control problem cleanly, with the cost-to-go surface reconstructing the underlying physics.

Building upon this foundation, in the next chapter, we will be exploring and learning about deep reinforcement learning.

The aim, as always, is to build a strong conceptual foundation and equip you with a flexible framework for reasoning about the core principles and the broader landscape in general.

As always, thanks for reading!

Any questions?

Feel free to post them in the comments.

Or

If you wish to connect privately, feel free to initiate a chat here:



A daily colour
practices on
professional:



A daily column with insights, observations, tutorials and best practices on python and data science. Read by industry professionals at big tech, startups, and engineering students.

chat icon



Menu

Contact

Published on May 24, 2026

FAQ

 Comments

 Share

Previous

< **Model-Free Learning**

Next

Introduction to Deep RL and DQN >



Daily Dose of Data Science © 2026